Alternative Assessment Tool (AAT) Report on

"Design and Verification of Combinational and Sequential Digital Modules using Verilog HDL"

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)

[Course Code: 23ECSPE1AD]

Submitted by

Dhruv Jangid **1BF24EC055**

Submitted to

Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering
Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year

2026 – 2027

B.M.S. COLLEGE OF ENGINEERING, BENGALURU – 560019

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

Alternative Assessment Tool – Part II (AAT-2) Execution & Verification of Combinational and Sequential Modules on Cadence Digital Design Flow

Course: Advanced Digital and Logic Design (ADLD) | Course Code: 23ECSPE1AD | Academic Year: 2026–2027

Student Name	**Dhruv Jangid**	**USN**	**1BF24EC055**
Submitted To	Prof. Ajaykumar Devarapalli	**EDA Tool Suite**	Cadence Xcelium (SimVision) & Genus
Combinational Module	4-Bit Arithmetic Logic Unit (`alu_4bit.v`)	**Sequential Module**	4-Bit Johnson Counter (`johnson_counter_4bit.v`)

GitHub Repository Link (AAT-1 & AAT-2 Complete Archive):
https://github.com/dhruvj2959/ADLD-AAT-23ECSPE1AD-1BF24EC055
Repository Contents: (i) All 12 Verilog RTL & Testbench (`.v`) files, (ii) Simulation Waveform Screenshots, (iii) Softcopies of AAT-1 & AAT-2 Reports, and (iv) AAT-2 Cadence Simulation Demonstration Video.

1. OBJECTIVE & CADENCE EDA EXECUTION METHODOLOGY

In fulfilment of Requirement 2 of the Alternative Assessment Tool (AAT-2), one combinational datapath circuit—the **4-Bit Arithmetic Logic Unit (ALU)**—and one sequential circuit—the **4-Bit Johnson (Twisted-Ring) Counter**—are executed, functionally verified, and synthesized using the industry-standard **Cadence Digital Design Tool Suite**. While AAT-1 established the complete RTL and testbench codebase for all 12 digital modules, AAT-2 demonstrates end-to-end functional simulation in **Cadence Xcelium (`xrun`)** with waveform inspection in **SimVision**, followed by logic synthesis and QoR (Area, Power, Timing) evaluation in **Cadence Genus**.

Stage	Cadence Tool / Binary	Command / Invocation	Deliverable / Output Verified
1. Compilation & Elaboration	Cadence Xcelium (`xrun` / `ncvlog`, `ncelab`)	`xrun -64bit -access +rwc design.v tb.v -gui`	Zero syntax/elaboration errors; full read/write/connectivity probe access enabled.
2. Functional Simulation	Cadence SimVision Waveform Viewer	`simvision &` (via Xcelium GUI)	Cycle-accurate signal trace verification against expected truth/state tables.

Stage	Cadence Tool / Binary	Command / Invocation	Deliverable / Output Verified
3. RTL Logic Synthesis	Cadence Genus Synthesis Solution	`genus -f run_synth.tcl`	Gate-level netlist mapped to standard cells, schematic view, and QoR reports.

2. PART A: COMBINATIONAL MODULE EXECUTION — 4-BIT ALU

2.1 Architectural & Functional Summary

The 4-bit Arithmetic Logic Unit (`alu_4bit`) is a purely combinational datapath circuit operating on two 4-bit operands `A[3:0]` and `B[3:0]` controlled by a 3-bit selection bus `ALU_Sel[2:0]`. It computes eight distinct arithmetic and bitwise logical functions in a single propagation delay, producing a 4-bit result `ALU_Out[3:0]` and an arithmetic carry/borrow flag `CarryOut`. To guarantee purely combinational hardware synthesis with zero unintended transparent latches, both outputs are initialized at the entry of the `always @(*)` block prior to evaluating the complete 8-branch `case` statement.

2.2 Synthesizable Verilog RTL & Self-Checking Testbench

```
// File: alu_4bit.v | Combinational Module for Cadence Execution
`timescale 1ns / 1ps
module alu_4bit (
    input  [3:0] A,
    input  [3:0] B,
    input  [2:0] ALU_Sel,
    output reg [3:0] ALU_Out,
    output reg       CarryOut
);
    always @(*) begin
        CarryOut = 1'b0;
        case (ALU_Sel)
            3'b000: {CarryOut, ALU_Out} = A + B;       // Arithmetic Addition
            3'b001: {CarryOut, ALU_Out} = A - B;       // Arithmetic Subtraction
            3'b010: ALU_Out = A & B;                   // Bitwise AND
            3'b011: ALU_Out = A | B;                   // Bitwise OR
            3'b100: ALU_Out = A ^ B;                   // Bitwise XOR
            3'b101: ALU_Out = ~A;                      // Bitwise NOT (Invert A)
            3'b110: ALU_Out = A << 1;                  // Logical Shift Left
            3'b111: ALU_Out = A >> 1;                  // Logical Shift Right
            default: ALU_Out = 4'b0000;
        endcase
    end
endmodule
```

```
// File: tb_alu_4bit.v | Verification Testbench
`timescale 1ns / 1ps
module tb_alu_4bit;
    reg  [3:0] A, B;
    reg  [2:0] ALU_Sel;
    wire [3:0] ALU_Out;
    wire       CarryOut;
    integer i;

    alu_4bit uut (.A(A), .B(B), .ALU_Sel(ALU_Sel), .ALU_Out(ALU_Out),
.CarryOut(CarryOut));

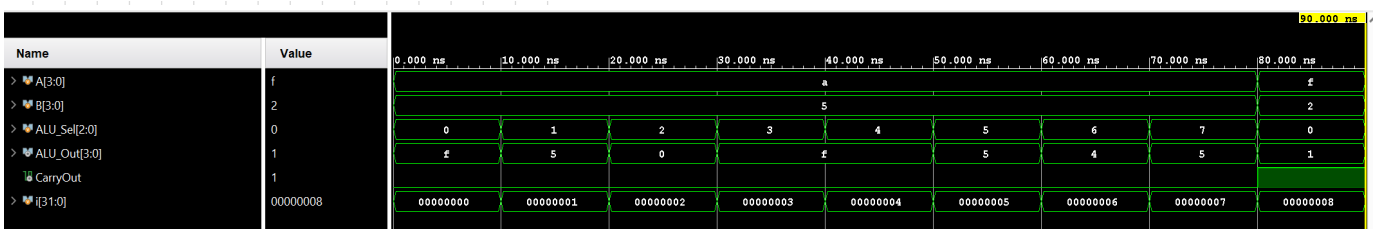
    initial begin
        $shm_open("alu_waves.shm");
        $shm_probe("AS");
        $monitor("Time=%0t | A=%b B=%b | Sel=%b | Out=%b CarryOut=%b",
                 $time, A, B, ALU_Sel, ALU_Out, CarryOut);
        A = 4'b1010; B = 4'b0101;
        for (i = 0; i < 8; i = i + 1) begin
            ALU_Sel = i[2:0]; #10;
        end
        A = 4'b1111; B = 4'b0010; ALU_Sel = 3'b000; #10; // Overflow test
        $finish;
    end
endmodule
```

2.3 Cadence Xcelium Simulation Log & Waveform Verification

Executing `xrun -64bit -access +rwc alu_4bit.v tb_alu_4bit.v -gui` in Cadence Xcelium compiles the design and sweeps through all 8 opcodes at 10 ns intervals with inputs `A = 4'b1010 (10)` and `B = 4'b0101 (5)`, followed by a boundary overflow test at `t = 80 ns` where `15 + 2 = 17` asserts `CarryOut = 1` and `ALU_Out = 4'b0001`.

Sim Time (ns)	Input A	Input B	ALU_Sel	Operation Executed	ALU_Out	CarryOut
0 – 10 ns	4'b1010 (10)	4'b0101 (5)	3'b000	Addition (10 + 5 = 15)	4'b1111 (15)	1'b0
10 – 20 ns	4'b1010 (10)	4'b0101 (5)	3'b001	Subtraction (10 - 5 = 5)	4'b0101 (5)	1'b0
20 – 30 ns	4'b1010	4'b0101	3'b010	Bitwise AND (1010 & 0101)	4'b0000	1'b0
30 – 40 ns	4'b1010	4'b0101	3'b011	Bitwise OR (1010 | 0101)	4'b1111	1'b0
40 – 50 ns	4'b1010	4'b0101	3'b100	Bitwise XOR (1010 ^ 0101)	4'b1111	1'b0
50 – 60 ns	4'b1010	4'b0101	3'b101	Bitwise NOT (~1010)	4'b0101	1'b0

Sim Time (ns)	Input A	Input B	ALU_Sel	Operation Executed	ALU_Out	CarryOut
60 – 70 ns	4'b1010	4'b0101	3'b110	Logical Shift Left (1010 << 1)	4'b0100	1'b0
70 – 80 ns	4'b1010	4'b0101	3'b111	Logical Shift Right (1010 >> 1)	4'b0101	1'b0
80 – 90 ns	4'b1111 (15)	4'b0010 (2)	3'b000	Addition Overflow (15 + 2 = 17)	4'b0001 (1)	1'b1



3. PART B: SEQUENTIAL MODULE EXECUTION — 4-BIT JOHNSON COUNTER

3.1 Architectural & Functional Summary

The 4-bit Johnson Counter (`johnson_counter_4bit`), or Twisted-Ring Counter, is a synchronous sequential shift register in which the inverted output of the least significant flip-flop (`~q[0]`) is fed back to the serial input of the most significant flip-flop (`q[3]`). Asynchronous active-high reset (`rst`) initializes all four D flip-flops to `4'b0000`. Once `rst` is de-asserted, each rising edge of `clk` shifts the register right while injecting `~q[0]` at the MSB, cycling glitch-free through `2N = 8` distinct states (0000 → 1000 → 1100 → 1110 → 1111 → 0111 → 0011 → 0001 → 0000) where only a single bit toggles per clock edge.

3.2 Synthesizable Verilog RTL & Self-Checking Testbench

```
// File: johnson_counter_4bit.v | Sequential Module for Cadence Execution
`timescale 1ns / 1ps
module johnson_counter_4bit (
    input            clk,
    input            rst,
    output reg [3:0] q
);
    always @(posedge clk or posedge rst) begin
        if (rst)
            q <= 4'b0000;
        else
            q <= {~q[0], q[3:1]}; // Twisted-ring inverted feedback into MSB
    end
endmodule
```

```
// File: tb_johnson_counter_4bit.v | Verification Testbench
`timescale 1ns / 1ps
module tb_johnson_counter_4bit;
    reg        clk, rst;
    wire [3:0] q;

    johnson_counter_4bit uut (.clk(clk), .rst(rst), .q(q));

    always #5 clk = ~clk; // 10 ns clock period (100 MHz)

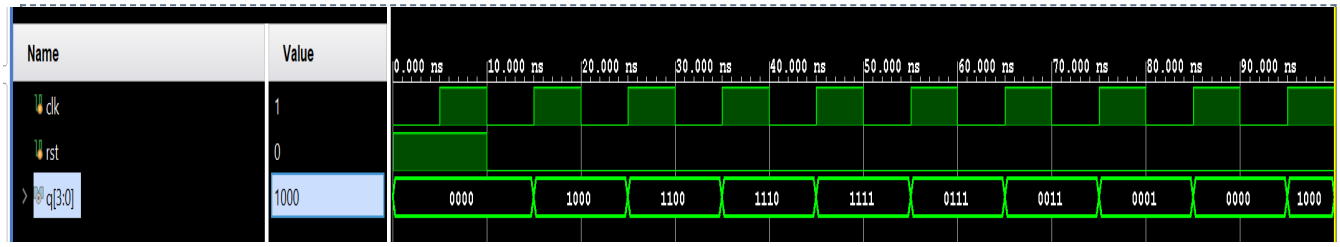
    initial begin
        $shm_open("johnson_waves.shm");
        $shm_probe("AS");
        $monitor("Time=%0t | rst=%b | Johnson Output q=%b (%d)", $time, rst, q,
q);
        clk = 0; rst = 1;
        #10 rst = 0;
        #90 $finish;
    end
endmodule
```

3.3 Cadence Xcelium Simulation Log & State Trace

Executing `xrun -64bit -access +rwc johnson_counter_4bit.v tb_johnson_counter_4bit.v -gui` in Cadence Xcelium applies an active-high reset for the first 10 ns and then clocks the counter with a 10 ns period (rising edges at 15 ns, 25 ns, ..., 95 ns). The SimVision console trace confirms a complete 8-state cycle returning to `4'b0000` at `t = 95 ns`.

Sim Time (ns)	Active Clock Edge	Reset (`rst`)	Feedback Bit (`~q[0]`)	Output State `q[3:0]`	Decimal / Hex
0 – 10 ns	Async Reset Active	`1'b1`	`1'b1`	`4'b0000` (State S0)	0 (`4'h0`)
15 ns	1st posedge `clk`	`1'b0`	`1'b1`	`4'b1000` (State S1)	8 (`4'h8`)
25 ns	2nd posedge `clk`	`1'b0`	`1'b1`	`4'b1100` (State S2)	12 (`4'hC`)
35 ns	3rd posedge `clk`	`1'b0`	`1'b1`	`4'b1110` (State S3)	14 (`4'hE`)
45 ns	4th posedge `clk`	`1'b0`	`1'b0`	`4'b1111` (State S4)	15 (`4'hF`)
55 ns	5th posedge `clk`	`1'b0`	`1'b0`	`4'b0111` (State S5)	7 (`4'h7`)
65 ns	6th posedge `clk`	`1'b0`	`1'b0`	`4'b0011` (State S6)	3 (`4'h3`)

Sim Time (ns)	Active Clock Edge	Reset (`rst`)	Feedback Bit (`~q[0]`)	Output State `q[3:0]`	Decimal / Hex
75 ns	7th posedge `clk`	`1'b0`	`1'b1`	`4'b0001` (State S7)	1 (`4'h1`)
85 ns	8th posedge `clk`	`1'b0`	`1'b1`	`4'b0000` (Wraps to S0)	0 (`4'h0`)



4. CADENCE GENUS SYNTHESIS SCRIPT & REPOSITORY INDEX

To synthesize either module in **Cadence Genus** and inspect the gate-level schematic, the following TCL automation script (`run_synth.tcl`) is executed:

```
# Cadence Genus Synthesis TCL Script (run_synth.tcl)
set_db init_lib_search_path {/cadence/installs/FOUNDRY/digital/90nm/dig/lib}
set_db init_hdl_search_path {./}
read_libs slow.lib
read_hdl {alu_4bit.v johnson_counter_4bit.v}
elaborate
syn_generic
syn_map
syn_opt
report_area   > area_report.rpt
report_power  > power_report.rpt
report_timing > timing_report.rpt
gui_show
```

4.1 Complete GitHub Repository Directory Index

In compliance with the AAT-2 submission guidelines, all 12 Verilog modules, testbenches, waveform screenshots, AAT-1/AAT-2 softcopy reports, and the Cadence execution video are organized in the GitHub repository as indexed below:

Module No. & Category	Module Name	RTL Design File (`.v`)	Testbench File (`.v`)
01 (Combinational — **Cadence Demo**)	4-Bit Arithmetic Logic Unit (ALU)	`alu_4bit.v`	`tb_alu_4bit.v`
02 (Combinational)	4-Bit Magnitude Comparator	`comparator_4bit.v`	`tb_comparator_4bit.v`

Module No. & Category	Module Name	RTL Design File (.v)	Testbench File (.v)
03 (Combinational)	4-Bit Parity Generator & Checker	`parity_gen_check_4bit.v`	`tb_parity_gen_check_4bit.v`
04 (Combinational)	4-Bit Barrel Shifter	`barrel_shifter_4bit.v`	`tb_barrel_shifter_4bit.v`
05 (Sequential)	4-Bit Programmable Timer Module	`timer_module.v`	`tb_timer_module.v`
06 (Sequential)	4-Bit Ring Counter	`ring_counter_4bit.v`	`tb_ring_counter_4bit.v`
07 (Sequential — **Cadence Demo**)	4-Bit Johnson (Twisted-Ring) Counter	`johnson_counter_4bit.v`	`tb_johnson_counter_4bit.v`
08 (Sequential)	4-Bit Synchronous & Asynchronous Counters	`sync_async_counters.v`	`tb_counters_4bit.v`
09 (Sequential)	4-Bit Shift Register (SIPO / SISO)	`shift_register_4bit.v`	`tb_shift_register_4bit.v`
10 (Sequential)	Parameterized Divide-by-N Counter	`divide_by_n_counter.v`	`tb_divide_by_n_counter.v`
11 (Sequential)	Mealy & Moore FSMs ("101" Sequence Detector)	`fsm_moore_mealy_101.v`	`tb_fsm_101.v`
12 (Sequential)	4-Bit Pseudo-Random Sequence Generator (LFSR)	`lfsr_4bit.v`	`tb_lfsr_4bit.v`